

Unit-1

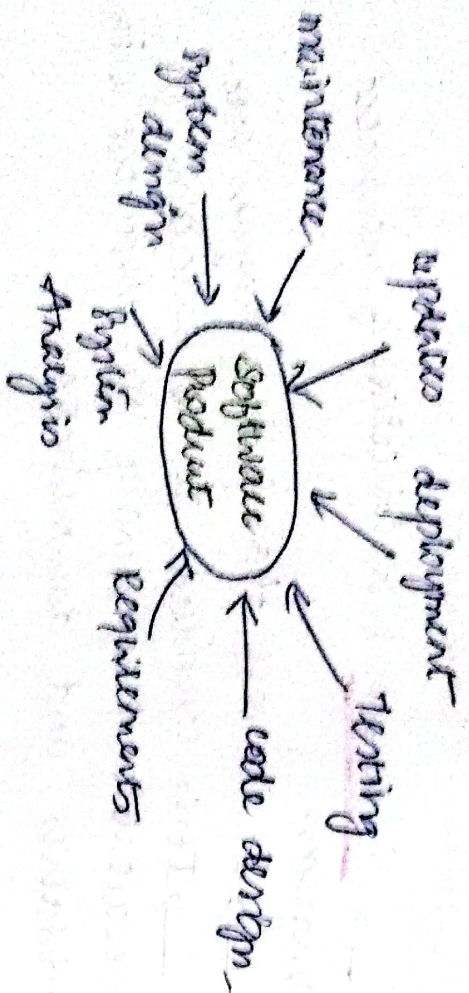
Software Engineering :-

- Software is a collection of integrated programs.
- It is more than just a program code.
- It is considered to be collection of executable programming code, associated with libraries and documentation.

Engineering :-

- It is a process of designing and building something that serves a particular process to find a cost-effective solution to problems.

→ It is a application of scientific and practical knowledge to invent, design, build, maintain & improve frameworks, processes etc.



Definition:-

IEEE Definition:-

→ The Application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software (i.e.) the application of Engineering to software.

→ Designing, Building and maintaining large software systems in a cost-effective way.

→ An Engineering approach to develop software.

→ It's a Engineering branch associated with development of software product using well-defined scientific principles, methods and procedures. includes Requirements analysis, design, testing + maintenance.

→ It's a process of designing, developing, testing and maintaining software.

→ It aims to create high-quality, reliable and maintainable software.

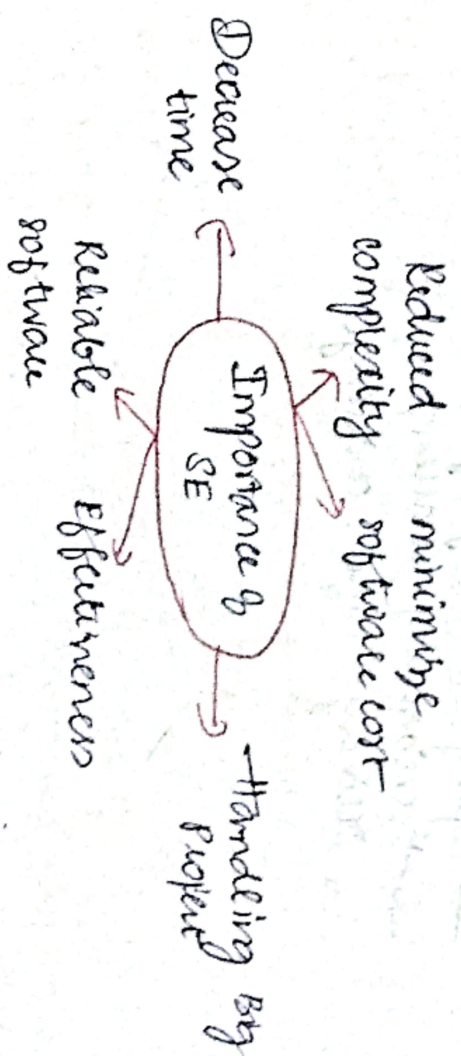
→ mainly used for large projects based on software systems rather than single program or Applications.

→ Ensures that the built software should be consistent; correct, Budget, on time and within the Required requirements.

Attributes of SE:-

- ① Efficiency
- ② Reliability
- ③ Robustness
- ④ Maintainability

Importance of SE:-



Various categories of software :-

- System software
- Application software
- Engineering/scientific software
- Embedded software
- Web Applications
- Artificial Intelligence software

Goals/Objective of SE:-

- Satisfy user Requirements
- High Reliability
- Low maintenance costs
- Delivery on time
- Low production costs
- High Performance
- Ease to reuse

Characteristics of software:-

- software is developed n Engineered
- it is, not manufactured.
- Software does not wear out (we can use it)
- Most software is custom components, built & then assembled from assembling the Engineering components (it is custom Built)

Why is software Engineering Required?

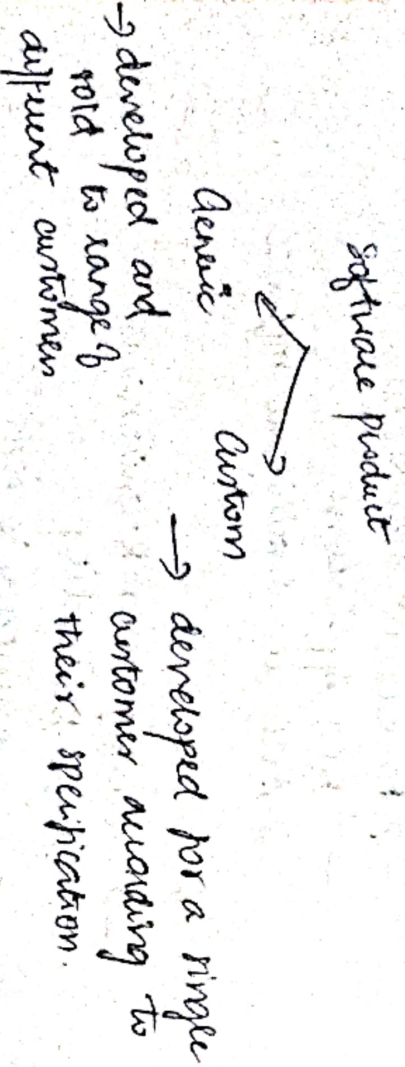
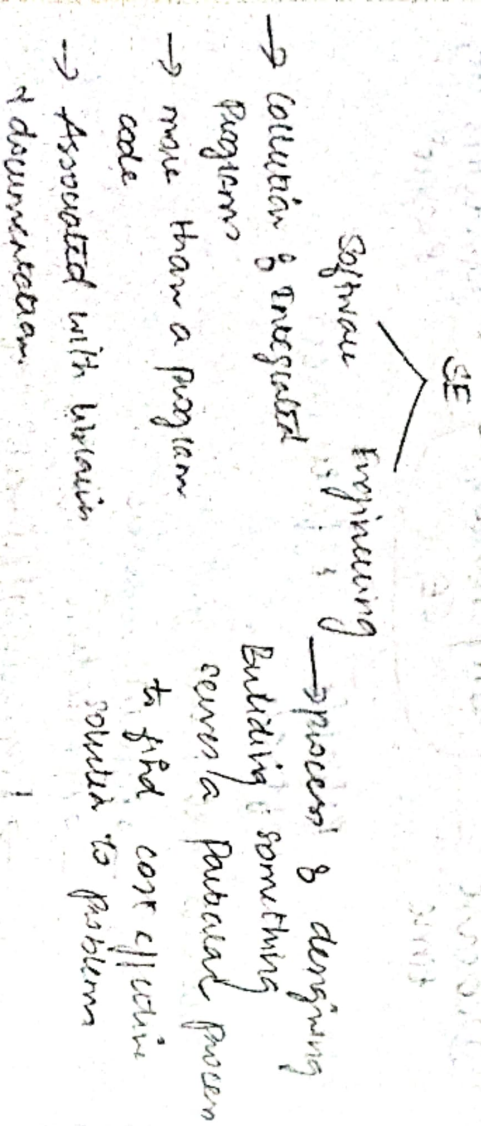
- To manage large software
- More scalability
- cost management
- to manage dynamic Nature of software
- For Better Quality management

Need for SE:-

- Huge Programming
- Adaptability
- cost
- Dynamic Nature
- Quality management

Challenges in SE:-

- coping with legacy systems
- Heterogeneity challenge
- Delivery time challenge



① Feasibility study → technical financial

② Requirement Analysis & specification:

R. Analysis R. Gathering

③ Design Approval

Traditional object oriented

structured structured Analysis design

high level design detailed design

④ coding & Unit testing

⑤ Integration & system testing

α Acceptance

⑥ maintenance

Software Engineering:-

- Software Engineering discusses systematic and cost-effective techniques for software development
- these techniques help develop software using an Engineering approach.

(or)
"An Engineering approach to develop software"

IEEE Definition for software Engineering:-

- the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.
- (i.e) the application of Engineering

software

- designing, building and maintaining large software systems in an cost-effective way.

Characteristics of software:-

- Software is Engineered, not manufactured.
- Software does not wear out
- Most software is custom built rather being assembled from components.

Various categories of software:-

- System software → collection of programs written to serve other programs (i.e) compiler, editor, assembler.
- Application software → standalone programs developed for specific business need, supported by database systems.
- Engineering/Scientific software → based on complex numeric computations (i.e) Astronomy to volcanology, etc...
- Embedded software → consists of programs that reside within a product or system used to implement a control function + functions for the end user + for system itself.
- Web Applications → consists of web pages that are Retrieved by a browser
Developed using programming languages like JAVA, HTML, ...
- Artificial Intelligence software → Based on knowledge based expert systems
used in Robotics, image + voice recognition etc.

Goals/Objectives of Software Engineering:-

- satisfy users requirements
- high reliability
- low maintenance costs
- Delivery on time
- low production costs
- high performance
- Ease of reuse

Challenges in Software Engineering:-

- Coping with legacy systems:-
old and valuable systems must be maintained and upgraded.

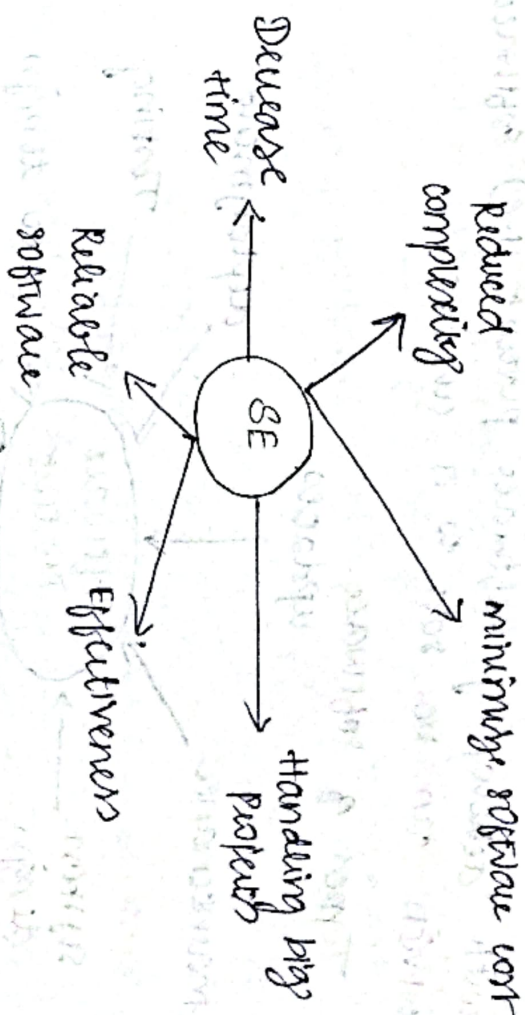
Hardware is faster than software
If original developer have moved on
managing, maintaining or integrating a software
becomes a critical issue.

→ Heterogeneity challenge:-

some systems are distributed which
includes the mix of hardware and software.
this implies software systems to
cleanly integrate with other different software
systems built by different organizations + teams

using different hardware and software platforms
→ Delivery time challenge
Increasing pressure for faster delivery
of software.

complexity of systems we develop
increases, this makes the challenge become harder
Importance of Software Engineering:-

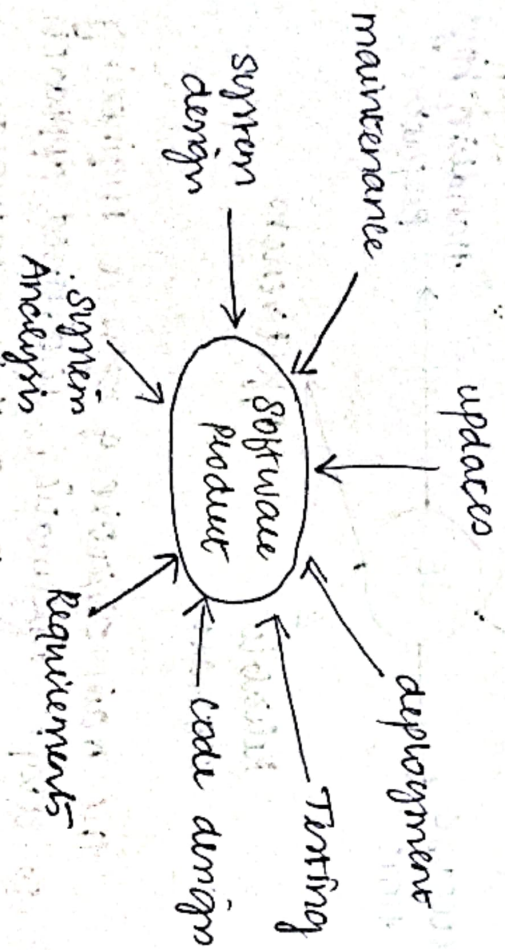


The main objective of software engineering
is to adopt systematic, disciplined approach
while building high quality software.

Software product:- Is the product that software

Engineers design and built to the customers. They are software systems being delivered to a customer with documentation which describes how to install & use the system.

Software Engineers are concerned with developing software product (i.e.) software which can be sold to a customer.



Types of software product:-

- Generic products
- custom products

Generic products:-

→ stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them.

→ Eg:- databases, word processors, drawing packages, and project management tools.

custom products:-

Systems that are commissioned by a particular customer. Software developer develops the software especially for that customer.

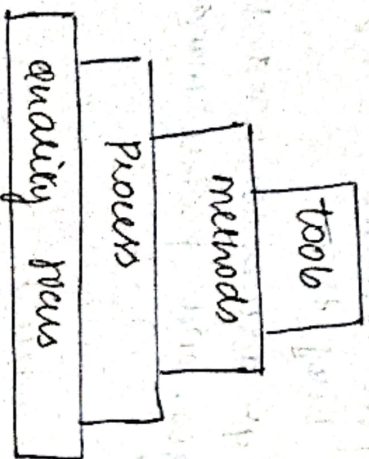
Eg:- control systems for electronic devices, systems to support a particular business process and air traffic control systems.

Layard Technology -

Software Engineering is a layered technology.

Any software can be developed using these layered approaches.

Various layers on which the technology is based are quality focus layer, process layer, methods layer, tools layer.



A disciplined quality management is a backbone of software engineering technology.

→ The principle that support software

Engineering is quality focus.

→ The foundation of software engineering is the process layer.

The software Engineering process is the glue that holds the technology layers together and enables rational and timely development

of complete software.

It defines the framework for timely delivery of software.

It forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products are produced, milestones are established, quality is ensured and change is properly managed.

→ In method layer, the actual method of implementation is carried out with the help of requirement analysis, designing, coding using desired programming constructs and testing.

It relies on set of basic principles that governs each area of the technology & include modeling, activities and other descriptive techniques.

→ Software tools are used to bring automation in software development process.

When tools are integrated, information created by one tool can be used by another, a system for the support of software development is called "computer-aided software Engineering".

Thus, software Engineering is a combination of process, methods, and tools for development of quality software.

Software process:-

Software process can be defined as the structured set of activities that are required to develop the software system.

The fundamental activities are:-

- Specification
- Design & implementation
- Validation
- Evolution

Software specification:-

Where customer and engineers define the software that is to be produced and constraints on its operation.

Software development:-

Where the software is designed and programmed.

Software Validation:-

Where the software is checked to ensure that it is what the customer requires.

Software evolution:-

Where the software is modified to reflect changing customer and market requirements.

A software process model is an abstract representation of a process.

It presents a description of a process from some particular perspective

A process is a collection of activities, actions and tasks that are performed when some work product is to be created.

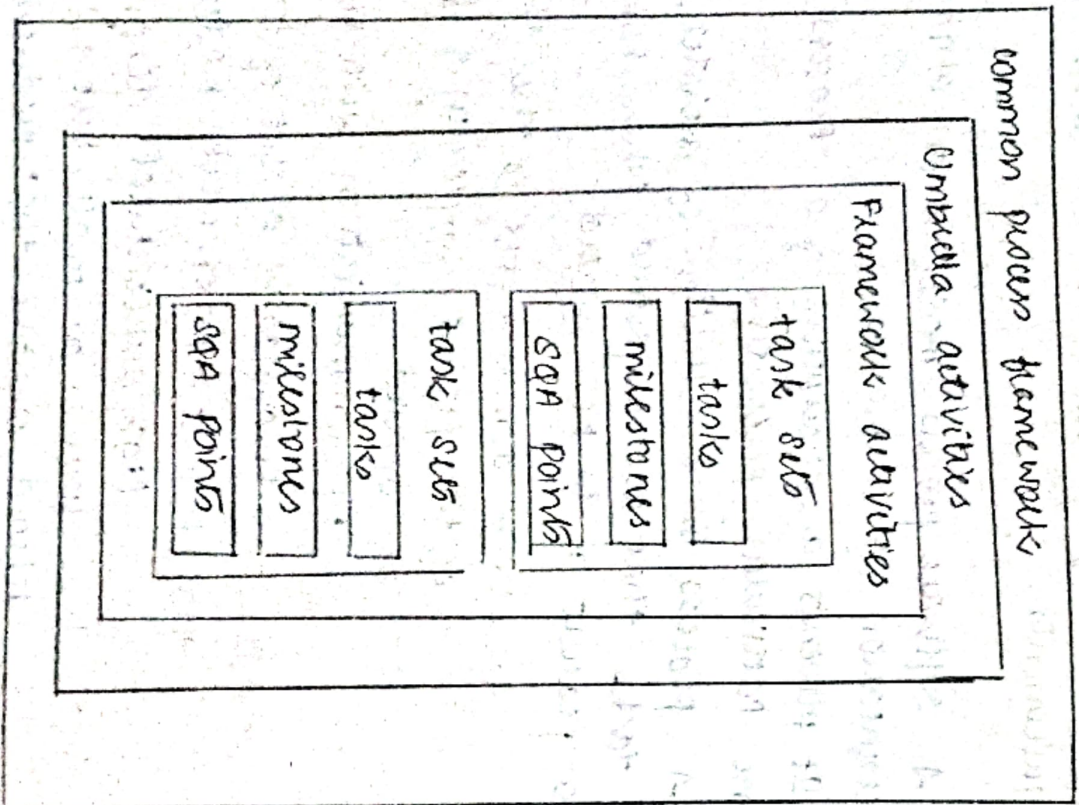
Activity — domain, size of the object of the project, complexity & effort to be applied

Eq:- communication with stakeholders

Action — Encompasses a set of tasks that produce major work product (Eq:- architectural design).

Task — focuses on small, but well-defined objective that produces a tangible outcome Eq:- conducting unit test

Common process frameworks -
 the process framework is required for representing the common process activities. Identifies small number of framework activities that are applicable to software projects regardless of the size or complexity.



The software process is characterized by process framework activities, task sets and umbrella activities.

5 Activities

- Communication
- Planning
- Modeling
- Construction
- Deployment

Communication → By communicating customer requirement gathering is done

Planning → Establishes Engineering work plan, describes technical risks, lists resource requirements, work products produced and defines work schedule.

Modeling → Software model is prepared by Analysis of requirements and design

Construction → software design is mapped into a code by code generation and testing.

Deployment → software delivered for customer evaluation + feedback is obtained.

Task set:-

Defines the actual work done in order to achieve the software objective.

used to adopt the framework activities & project team requirements using.

- collection of software engineering work tasks
- Project milestones
- software quality assurance points.

Umbrella Activities:-

It occurs throughout the process.

They focus on project management, tracking and control.

They are applied throughout a software project and help a software team manage and control progress, quality, change and risk.

The umbrella activities are

- software project tracking and control - allows the software team to assess the progress against the project plan and take any necessary action to maintain the schedule.

Risk management - Risks may affect the outcome of the project or quality of the product

Software quality assurance - Maintains the quality.

Technical reviews - Assess software Engineering work products to uncover and removes errors before they propagate to next activity.

Software configuration management - Task of tracking and controlling changes in the software development part.

Manages the effects of changes throughout the software process.

Documentation - Project planning and other activities should be traced copied.

Work product preparation and production -

used to create models, documents, logs, forms & lists are carried out.

Reusability management - defines the criteria for work product reuse.

Measurement - Process can be defined & collected. Project & product measures are used to assist software team in software delivery.

capability maturity model (CMM) used to develop

It is a methodology used to develop and define an organization's software development process.

used for improving the software

process

5 process maturity levels are

Initial → work is performed informally.

Repeatable → work is planned and tracked.

Defined → work is well defined.

Managed → work is quantitatively controlled.

Optimizing → work is based upon continuous improvement.

Evolutionary Model:-

Evolutionary Model is a combination of iterative and incremental Model of software development.

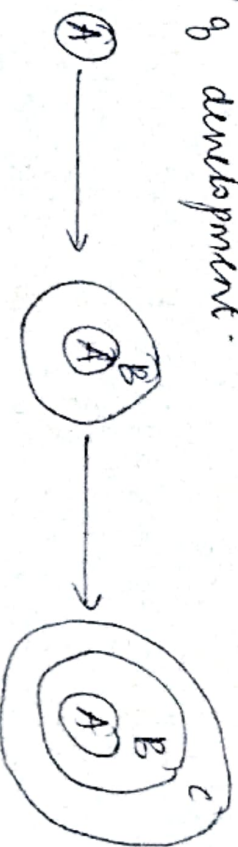
In Evolutionary Model, the software requirement is broken down into several modules that can be incrementally constructed and delivered.

Development first develops the core modules of the system.

Are those that do NOT need services from the other modules.

Initial product skeleton is refined into increasing levels of capability by adding new functionalities in successive versions.

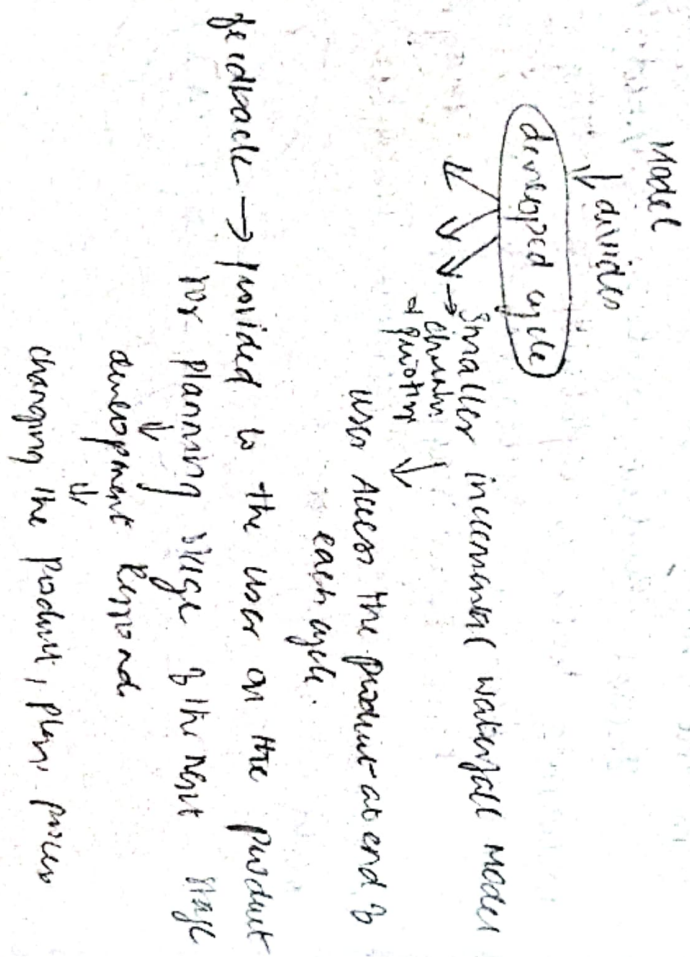
Each Evolutionary Model may be developed using an iterative waterfall model of development.



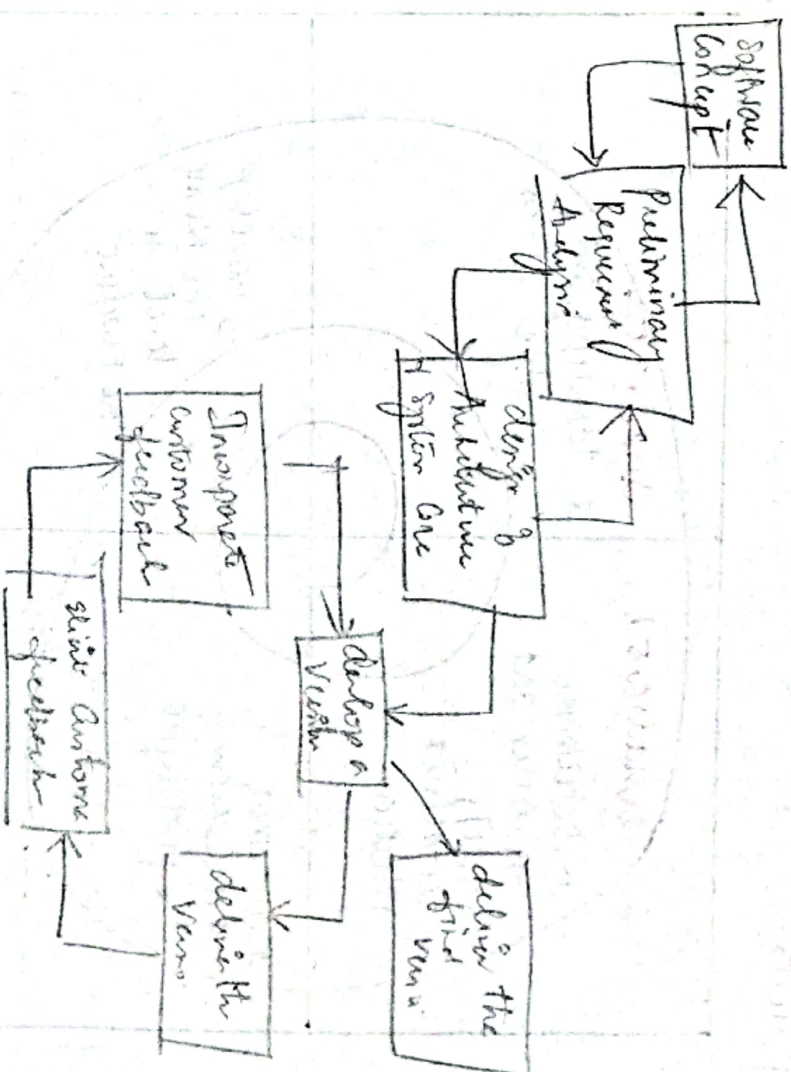
Each successive Model of the product is fully spanning software lifecycle of Programming more work than previous Model.

Normally, it is useful for very large products where it is easier to find modules for incremental implementation.

Evolutionary model is used when the customer prefers to receive the product in increments. So, that he/she can start using different features as and when they are developed rather than waiting all the time for full product to be developed & delivered.

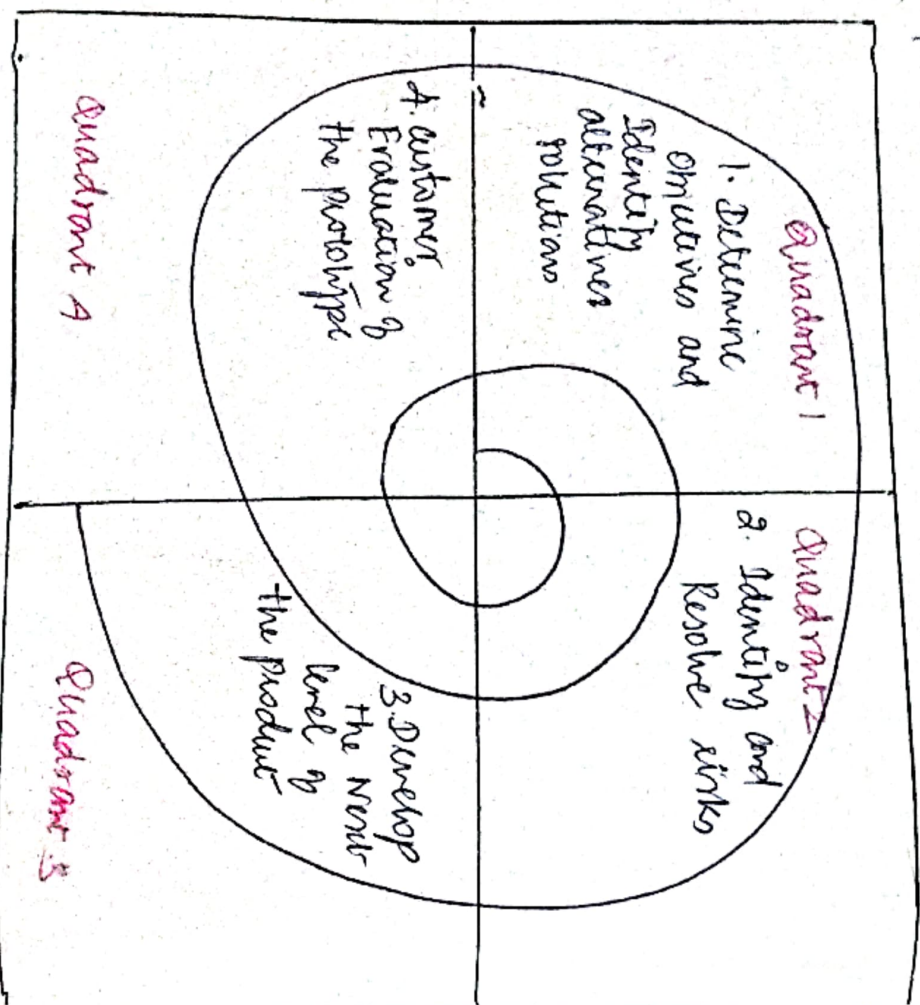


- Adv:-
- 1. customer confidence increases
 - he contracts get the goods from the beginning of the project to verify & validate his requirements
 - Allow for changing requirements
 - work is broken down into maintainable work chunk.



Spiral Model Software development

[Spiral Model is a system development lifecycle approach to software management that combines iterative used for the iterative development process model with elements of the waterfall model. It is used by software engineers and it is favored for large, expensive and complicated projects. limited.]



Each phase of a spiral model is divided into 4 quadrants.

Quadrant 1:- Waterfall Model

Requirements are gathered from the

customer.

Objectives are identified, elaborated and analysed at the start of every phase.

Then, alternative solutions possible for the phase are proposed in this quadrant.

Quadrant 2:- Prototype Model

All possible solutions are evaluated to select the possible solution. Then, the risks associated with that solution are identified and are resolved using the best possible strategy.

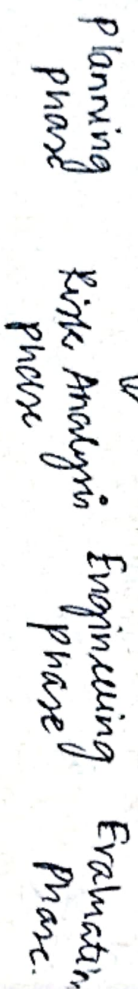
In the end, prototype is built for the best possible solution.

Quadrant 3:-

Identified features are developed and verified through testing.

In the end, next version of the software is available.

A spiral model has 4 phases:-



Spiral model is a risk-driven Model, which means it focus is on managing risk through multiple iteration of the software development process.

Planning → * Requirements are studied and gathered.

* Feasibility study.

* Reviews and demonstrations are done to smooth the Requirements.

Op:- Requirements Understanding documents.

→ finalized list of requirements.

Risk Analysis → Requirements are studied & problem-solving techniques are carried out to identify the potential risks.

Once, the risks are identified, risk mitigation strategy is planned & finalized.

Op:- Document which highlights all the risks and its mitigation plans.

Engineering → Actual development and testing.

Op:- code

→ test cases & results

→ test summary report & defect report.

Evaluation → customer evaluate the software & provides feedback and approval.

Op:- features implemented document.

Spiral model is a meta model:-

The spiral model is called a meta model because it subsumes all other SDLC models (discussed models).

Eq:- A single loop spiral actually represents the creative workspace model.

Spiral model incorporates repetitive

approach of the classical waterfall model.

Spiral model uses the approach of prototyping model by building a

prototype at the start of each phase as risk-handling technique.

Also, the spiral model is more unstructured as supporting the Evolutionary Model - the situation along the spiral can be considered as evolutionary levels through which the complete system is built.

Advantages:-
Risks are detected at every phase

Risks Handling -
Good for large projects

Flexibility in Requirements - change in Requirement is easier in later phase
Adaptability to customer - customer sees the development of the product at every phase
Iterative and incremental Approach -

Improved Quality and communication -
better communication & development in

Disadvantages:-

Complex - more complex than other models
Not suitable for small projects

Expensive - requires expert people
Too much dependencies in Risk Analysis

Difficulty in time management - NO. of phases is unknown at start of the project

Complexity - involves multiple iterations to develop a time-consuming. -> Requires multiple

Review - Extensive -> Requires multiple iterations & Review

Planning, Risk Analysis & Evaluation -> Requires significant investment in

Comparison of different type cycle models

-> Classical waterfall Model:-

-> Basic model and all other type cycle models are the enhancement of this model.

-> Cannot used to be used in practical development projects.

-> Since, it supports no mechanism to correct the errors that are committed during any of phases but detected at later phase.

-> This problem is overcome by iterative waterfall model by providing feedback paths.

Iterative waterfall Model:-

-> widely used software development

-> simple to understand and use.

-> Suitable only for well-understood problems

-> Not suitable for development of large projects and for projects that suffer from large number of risks.

Prototyping Model:-

→ Suitable for projects in which user requirements or underlying technical aspects are not well understood.
However, all risks can be identified before project starts.

→ Popular for development of user interface part of projects.

Evolutionary Model:-

→ Suitable for large problems which can be decomposed into set of modules for incremental development and delivery.

→ Widely used for object-oriented development projects.

→ This Model can only be used if incremental delivery of the system is acceptable to the customer.

Spiral Model:-

→ Turned as meta Model, since it encompasses all other life cycle models.

→ Flexibility and Risk handling are inherently built into this Model.